Scenario 2:

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

Class Appliance {
private:
    string name;
    double powerRating;
    double hoursUsed;
    bool state;

public:
    Appliance(string applianceName, double rating, double
Hours = 0.0) {
        name = applianceName;
        powerRating = rating;
        hoursUsed = hours;
```

Dated:

```
    state = false;
}
void turnOn() { state = true; }
void turnOFF() { state = false; }
void toggle() { state = !state; }

void displayStatus() const {
    cout << "Appliance : " << name << " |
state: " << (state ? "ON" : "OFF")
    << " | Power Rating :" << powerRating << "W |
Hours Used : " << hoursUsed << "hrs" << endl;
}
double calculateEnergy(double actualPower)
const {
    return (actualPower * hoursUsed) / 1000.0;
}
string getName() const { return name; }
double getPowerRating() const { return powerRating; }
double getHoursUsed() const { return state; }
void setHoursUsed(double h) { hoursUsed = h; }
};
Class Fan {
private:
int speed;
int maxspeedLevels;
Appliance appliance;

public:
Fan(string Name, double basePower, int maxspeed,
                                    double hours);
```

Dated:

```
Appliance(name, basePower, hours), speed(1),
maxSpeedLevels(maxSpeed) {}

void setSpeed(int s) {
    if (s >= 1 && s <= maxSpeedLevels)
speed = s;
}
void increaseSpeed() { if (speed < maxSpeedLevels
speed++; }
void decreaseSpeed() { if (speed > 1) speed--;

class Bulb {
private:
    string bulbType;
    double brightnessLevel;
    Appliance appliance;

public:
    Bulb(string name, double ratedPower, string type, double bright
double hours) : appliance(name, ratedPower, hours),
bulbType(type), brightnessLevel(brightness) {}
=b; }

void increaseBrightness() { :}
(brightnessLevel <= 90) brightnessLevel += 10; }
void decreaseBrightness() { :}
(brightnessLevel >= 10) brightnessLevel -= 10; }
double ~~void~~ calculatePowerConsumption() const {
~~[redacted]~~
return appliance.getPowerRating() *
```

Dated:

```
        (brightnessLevel / 100.0);
    }
    void displayBulbDetails() const {
        appliance.displayStatus();
        cout << "- Bulb Type: " << BulbType << "\nBrightness: "
             << brightnessLevel << "%" << endl;

        cout << "- Actual Power Consumption: " << calculatePower
        Consumption() << "W" << endl; }

        Appliance& getAppliance() { return
        appliance; }
    };

class AC {
private:
    double desiredTemp;
    double roomTemp;
    Appliance appliance;

public:
    AC(string name, double ratedPower, double rTemp, double dTemp,
       double Hours) : appliance(name, ratedPower, hours),
roomTemp(rTemp), desiredTemp(dTemp) { }

    void setTemperature(double rTemp, double dTemp) {
        roomTemp = rTemp;
        desiredTemp = dTemp; }
```

Dated:

```
double calculateCoolingDifference() const {
return roomTemp - desiredTemp; }

double calculatePowerConsumption() const {
    double tempDiff =
calculateCoolingDifference();
    return appliance.getPowerRating() *
(tempDiff * 50.0); }

    void displayACDetails() const {
        appliance.displayStatus();

                                    }

int main() {
    Bulb* bulb1 = new Bulb("Living Room Bulb", 100.0
"LED", 70.0, 4.0);
    Bulb Bulb2("Bedroom Bulb", 60.0, "CFL", 50.0, 1.0)
Fan    fan1("Ceiling Fan", 50.0, 5, 5.0);
AC     ac1("Inverter AC", 1200.0, 30.0, 24.0, 2.0);

    bulb1->getAppliance().turnON();
    bulb2.getAppliance().turnON();
    fan1.getAppliance().turnON();
    ac1.getAppliance().turnON;

    fan1.setSpeed(3);
    bulb1->setBrightness(80.0);
    ac1.getTemperature(19.0, 22.0);
```

Dated: _____

```
cout << "\n :: Detailed Appliance status & Consumption :: " <<
                                                    endl;
    bulb 1 -> display Bulb Details();
    bulb 2 . display Bulb Details();
    fan 1 . display Fan Details();
    ac1 . display AC Details();

    double tariff = 60.0;

    double energy 1 = bulb -> getAppliance() . calculate Energy
    (bulb 1 -> calculate Power Consumption() );
        double energy Fan =
    fan 1 . getAppliance() . calculate Energy (Fan 1 . calculate Power
                                                Consumption());
        double energy AC :
    Aac1 . get Appliance() . calculate Energy ( ac1 . calculate Power
                                                Consumption());
    cout << "\n = Energy Consumption and Costs :: " << endl;
cout << bulb 1 -> getAppliance() . getName() << " -> " << energy 1
                                        * tariff << endl;
cout << bulb 2 . getAppliance() . getName() << " -> " << energy 2 <<
                    " kWh | cost : Rs. " <<
        energy 2 * tariff << endl;
    cout << fan 1 . get Appliance() . getName() << " -> " <<
energy Fan << " kWh | cost : Rs. " << energy Fan *
        tariff << endl;

double total Energy = energy 1 + energy 2 + energy Fan +
                                            energy AC;
    double total Cost = total Energy * Tariff ;
```

Dated: _____

```
cout << "\n Total household Energy Consumption : "
<< totalEnergy << " kWh " << endl;
    cout << "Total Electricity Bill : Rs. " << totalCost <<
                                                endl;
cout << "\n Toggling Appliance " << endl;
    bulb1 -> getAppliance().toggle();
    fan1.getAppliance().toggle();

    bulb1 -> displayBulbDetails();
    fan1.displayFanDetails();

    delete bulb1;
    return 0;
}
```